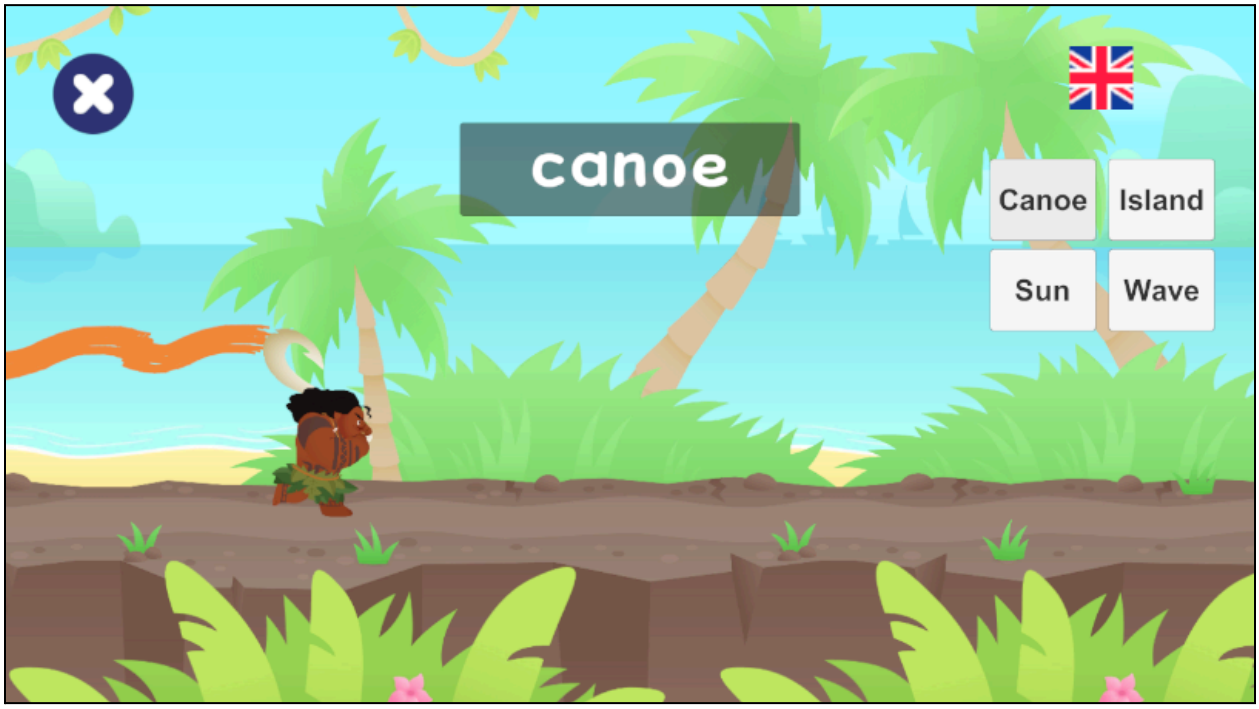
Exercise 2 – *Text Pill Framework* Documentation



Author: Macià Torrens (maciatp@gmail.com)
Role: Senior Technical Level Designer (Localization) Candidate

---

Index

1. Overview
2. Features
3. Component Structure
4. Inspector Parameters
5. Demo Setup
6. Usage
7. Localization Integration
8. Design Considerations
9. Limitations
10. Possible Extensions
11. Conclusion

1. Overview

The ***Text Pill Framework*** is a **reusable UI component** designed to **display short messages** (words or sentences) for a **limited duration**. It is intended for use in ***Lingokids*** activities to present vocabulary, feedback, or short instructions in a **clear and non-intrusive way**.

The component focuses on **simplicity**, **readability**, and **smooth transitions**, making it suitable for **young audiences**.

---

2. Features

- Displays **short text messages** with smooth fade and vertical motion.
- **Configurable timing** and **animation** parameters.
- Fully integrated with **Unity Localization Package**.
- Supports **runtime content updates.**
- Designed to integrate with **localization workflows**.
- Supports **dynamic global language switching**.
- Layout **adapts to different word lengths** using TextMeshPro auto-size.

---

3. Component Structure

```
TextPillFramework (Prefab Root)
    ├── TextCanvas
        ├── TextPill (Controller)
            ├── Background (Image)
            └── Label (TextMeshProUGUI)
```

- *CanvasGroup* controls fade transitions.
- *RectTransform* controls vertical movement.
- *TextMeshProUGUI* ensures readability and scalability.

---

4. Inspector Parameters

- ***Seconds Showing***
  Duration the text remains visible before hiding.
- ***Animation Duration***
  Controls the speed of fade and movement transitions.

- ***Hidden Y Offset***
  Vertical offset used to position the element outside the screen before and after animation.
- ***Canvas Group***
  Reference used to control alpha transitions.
- ***Rect Transform***
  Reference used for position-based animation.

---

5. Demo Setup

The demo scene (Assets/Project/Scenes/*DemoScene*) includes **multiple buttons** as presets, that assign **different localization** keys at runtime before triggering the Text Pill animation.

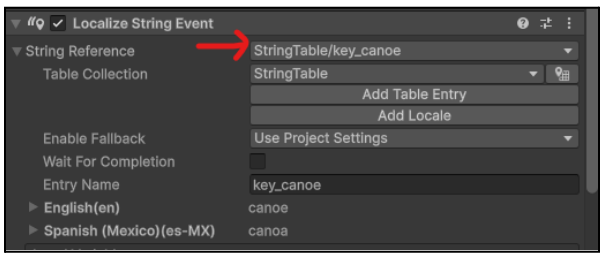
Additionally, the **system supports dynamic language switching**, allowing validation of the component across **different languages during runtime**.

This setup allows **quick validation** of the component with multiple inputs and simulates **real usage in a gameplay context**.

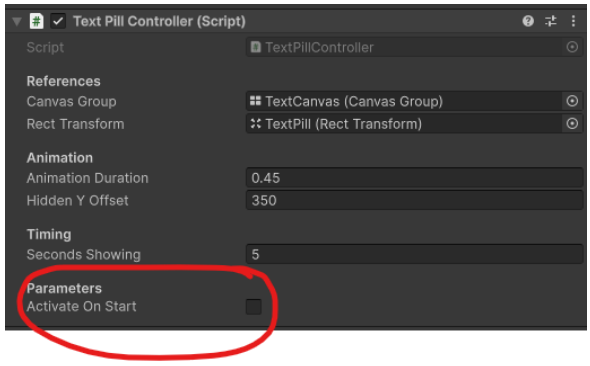
6. Usage

1. Add the *TextPillFramework* prefab to the scene. There are **two methods** of setting the **word to be displayed**, and **two methods** of playing the **animation**.

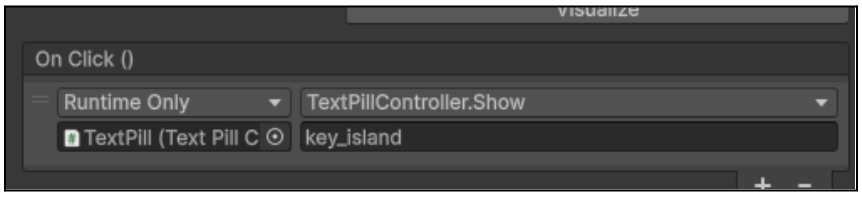
   a. To **change** the **word to be displayed** directly in the **Inspector**, use the *Localize String Event* component, attached to the LabelText gameobject to **select** one of the preset **localization keys**.



   b. Enabling the *ActivateOnStart* parameter on *TextPillComponent* plays the animation when the scene enters in *Play Mode* with the pre-set localization key.



2. The **second method** is to **change** the **localization key** during **runtime**. To do it, reference the ***TextPillController*** Script from a *UI button*.



3. Configure the *OnClick()* event the component using:

   textPillController.Show(*[selected_key]*); where *[selected_key]* can be:

   a. key_island
   b. key_canoe
   c. key_sun
   d. key_wave

4. **Global language switching system** is available through the button on the top-right corner of the *Game View* during *Play Mode*:

---

7. Localization Integration

The component is designed to **work with localized** content **without** directly **managing language data**.

Using the ***Localization Package***, a simple localization table has been implemented (English / Mexican Spanish) with a reduced dataset (4 entries) to demonstrate the workflow while keeping the scope concise.

Example:

Key: key_canoe
EN: Canoe
ES_MX: Canoa

At runtime, UI buttons assign different **localization keys** before triggering the Text Pill. This **simulates how the component would receive localized content** dynamically from an **external system**.

The ***Text Pill Framework*** focuses exclusively on **presentation** (timing, animation, readability), while localization **logic remains external**. This **separation** ensures **scalability** and **flexibility** across **multiple languages**.

---

8. Design Considerations

- The **timer is not visible**, keeping the UI clean and avoiding unnecessary on-screen elements that could **distract the player**.
- **Smooth easing** is applied to animations to improve **readability** and **comfort**.
- The component **prioritizes clarity** over complexity.
- Text **length variability** is handled using TextMeshPro auto-size, allowing the layout to adapt to **different word lengths**.
- The system is designed to support **dynamic content injection**.

---

9. Limitations

- The localization system is intentionally **simplified and limited in scope**, although it **uses** all the **necessary tools of a real product**.

---

10. Possible Extensions

- Support for **audio playback** (word pronunciation).
- Additional **visual themes** (success, error, hint).
- Use a **Google Spreadsheet extension** to use an **external database** and enable **simultaneous collaboration**.

---

11. Conclusion

The Text Pill component demonstrates a **reusable and scalable UI element** designed for educational gameplay. It separates presentation from content, supports **runtime updates**, and integrates with a **simplified localization workflow**.

The inclusion of a lightweight **localization table** and **runtime key assignment** shows how the component can operate within a **real production pipeline** while maintaining a **focused and concise implementation**.